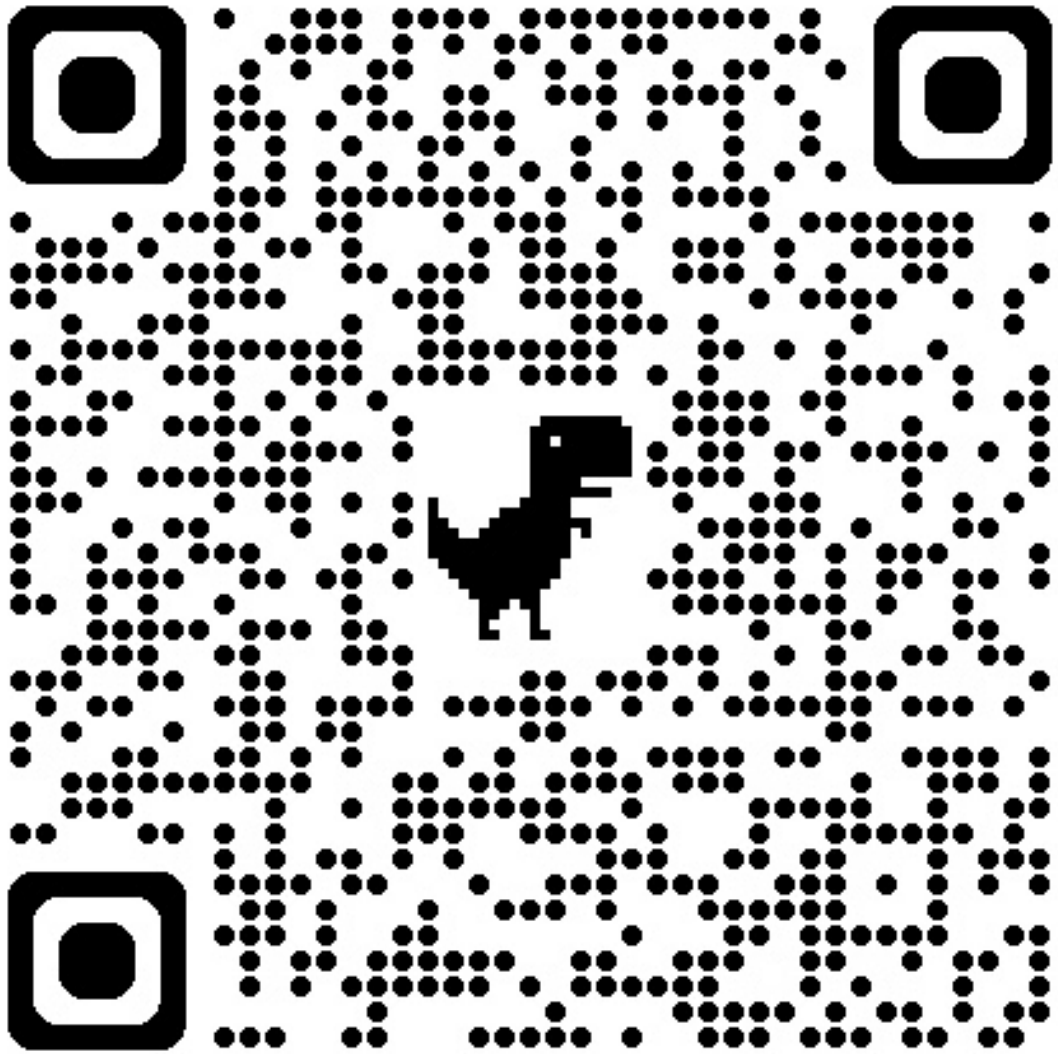
FILE READING, NESTED FOR, AND LIST COMP (LECTURE)



FOR LOOP PRACTICE

What is the value of `skills` after this code is run? **(S7)**

```
skills = [18, 88, 20, 82, 91, 78, 15]
for i in range(len(skills) - 1):
    if skills[i] >= 20:
        skills[i] = skills[i] + 7
    else:
        skills[i] += 2
```

If you already did this last time: can you modify the code above so that it builds a new list instead?

WHAT DOES THIS DO? (THINK, CONFER, DISCUSS)

```
strings = ["My", "Anti", "Aircraft", "Friend"]

longest_str = strings[0]

for string in strings:
    if len(string) > len(longest_str):
      longest_str = string

print(f"The longest string is {longest_str}.")
```

1. What value would be printed at the end?

2. If you were going to add comments to this program, what would you add?

WHAT DOES THIS DO? (THINK, CONFER, DISCUSS)

```
strings = ["My", "Anti", "Aircraft", "Friend"]
# To start, guess that the first string is the longest.
# Might be right or wrong, but we need to start somewhere.
longest_str = strings[0]

# Visit each string in the list one by one...
for string in strings:
    # if the current string is longer than the biggest we've seen so far...
    if len(string) > len(longest_str):
      # ...write down the current string as the new biggest so far.
      longest_str = string
    # if the current string is not bigger, do nothing and keep going in loop.

# after we look at all strings, the longest "so far" is the longest overall.
print(f"The longest string is {longest_str}.")
```

REVIEW: FOR LOOPS W/ ENUMERATE()

We can use `enumerate()` to get the indices and items of the sequence paired together:

```
nums = [3, 2, 5]
for idx, item in enumerate(nums):
    print(f"Index {idx}: {item}")
```

```
Index 0: 3
Index 1: 2
Index 2: 5
```

Each iteration of the loop gives us two values to work with, so we choose two variable names.

ENUMERATION PRACTICE

We want to write some code to find *the index of the longest string* in a list. Fill in the loop body. C12

```
strings = ["My", "Anti", "Aircraft", "Friend"]

index_of_longest = 0
longest_str = strings[0]

for i, string in enumerate(strings):
    # TODO: Fill out this loop




print(f"The longest string is {longest_str} at index {index}")
```

FILES

FILES

On computers we have things called **files**.
Files are where we store information that the computer can still access even after the computer turns off and on again.

We have already use files before: our programs are stored in `.py` files.

For now, we can assume that the contents of files are all characters. For text files like these, we think of files as being made of a "sequence" of lines of text.

```
Is there anybody     # first line
                     # second line
out there?           # third line
```

OPEN() AND CLOSE()

To read a file, we need to create a file "object" associated with that file. We can create a variable holding a file object with the `open()` call.

```
# opens the file "filename.txt" with "r" (Reading) enabled
example_file = open("filename.txt", "r")
```

When we are completely done with a file, we need to close it

```
example_file.close()
```

...but what do we do in between the opening and closing?

READLINE()

Use `readline()` to read a line from the file.

```
# first_three_lines.py
import sys
my_file = open(sys.argv[1], "r") # 🤔 what was this sys.argv thing? 🤔
for i in range(3):
    line = my_file.readline()
    print(line)
my_file.close()
```

The next time we call `readline()` we get the next line of the file. These File objects remembers our position in the file.

TRY IT! `python first_three_lines.py hello.txt`

PRACTICE:

Assume we have a file named `beep.boop` with the layout:

```
this file has 3 lines after this
line 0
line 1
line 2
```

Write some code that can read a file like this and print out all the lines but the first. Assume that the file can have any number instead of `3`. **(C16)**

- You should probably use: `open(filename, "r")`, `file.readline()`, `file.close()`, `string.strip()`, `string.split()`, and `int()`

NESTED FOR

We can have loops inside of loops

What does this print? S8

```
for e in range(1, 4):
    prod = 1
    for x in range (1, e + 1):
        prod = prod * x
    print(prod)
```

NESTED FOR: TRACING TABLE

E	PROD (BEFORE INNER LOOP)	X	PROD (AFTER ITERATION OF INNER LOOP)	PRINT(PROD)
1	1	1	1 × 1 = 1	1
2	1	1	1 × 1 = 1	
		2	1 × 2 = 2	2
3	1	1	1 × 1 = 1	
		2	1 × 2 = 2	
		3	2 × 3 = 6	6

Output: 1, 2, 6 (This calculates factorials!)

LIST COMPREHENSION SYNTAX

Recall a `for` loop that copies all characters of a string into a list:

```
new_list = []
for character in "ABCD":
    new_list.append(character)
```

"For each character in the string, place that character in the new list I am creating."

👇 👇 👇

```
new_list = [character for character in "ABCD"]
```

LIST COMPREHENSION SYNTAX

```
[<expression> for variable in sequence]
```

- `for variable in sequence` works exactly like a regular `for` loop
  - Each element in `sequence` gets visited one-by-one and is given the name `variable`
- The value of `<expression>` is appended to the output list for each element in the sequence
  - Usually write `<expression>` in terms of `variable`
- A new list is created!

RECALL: GETTING NON-ZERO EXAM SCORES

```
exam_scores = [100, 0, 89, 93, 78, 67, 0]
non_zeroes = []                 # [] is a list with no contents
for score in exam_scores:       # For each score from the list,
  if score > 0:                 # if that score is not zero,
    non_zeroes.append(score)    # add that score to the end of the new list.
```

...can be rewritten to:

```
exam_scores = [100, 0, 89, 93, 78, 67, 0]
non_zeroes = [score for score in exam_scores if score > 0]
print(non_zeroes)
```

```
[100, 89, 93, 78, 67]
```

LIST COMPREHENSION PRACTICE L11 + L12

Fill out the list comprehension so we have a list with each element of `values` with 10 added to it.

```
values = [0, 5, 10, 23]
values_added_ten = [# TODO: fill in this list comprehension]
```

Write a list comprehension that gets a list of all even length strings from `names`:

```
names = ['bob', "steve", "pete", "me", "abcde"]
even_naes = [ _______________ ]
```

more practice on the next slide

MORE PRACTICE L13

Write the equivalent of this code:

```
strings = ["arriving", "somewhere", "but", "not", "here"]
result = []
for i,string in enumerate(strings):
    new_entry = (' '*i)+string
    result.append(new_entry)
```

but use a list comprehension, e.g.:

```
strings = ["arriving", "somewhere", "but", "not", "here"]
result = [
    # TODO: fill in this list comprehension
]
```

REMINDER:

- There is another check-in due before lecture as always.
- We highly recommend you look back at some of these lecture examples for the HW02